

# 手机App防沉迷系统

## 题目描述

智能手机方便了我们生活的同时，也侵占了我们不少的时间。

“手机App防沉迷系统”能够让我们每天合理地规划手机App使用时间，在正确的时间做正确的事。

它的大概原理是这样的：

1. 在一天24小时内，可以注册每个App的允许使用时段



2. 一个时间段只能使用一个App，举例说明：不能同时在9:00-10:00注册App2和App3



3. App有优先级，数值越高，优先级越高。注册使用时段时，如果高优先级的App时间和低优先级的时段有冲突，则系统会自动注销低优先级的时段，如果App的优先级相同，则后添加的App不能注册。

举例1：

(1) 注册App3前



(2) App3注册时段和App2有冲突



(3) App3优先级高，系统接受App3的注册，自动注销App2的注册



举例2:

(1) 注册App4



(2) App4和App2及App3都有冲突，优先级比App2高，但比App3低，这种场景下App4注册不上，最终的注册效果如下



4. 一个App可以在一天内注册多个时段



请编程实现，根据输入数据注册App，并根据输入的时间点，返回时间点使用的App名称，如果该时间点没有注册任何App，请返回字符串 “NA”。

输入描述

输入分3部分：

- 第一行表示注册的App数量  $N$  ( $N \leq 100$ )
- 第二部分包括  $N$  行，每行表示一条App注册数据
- 最后一行输入一个时间点，程序即返回该时间点使用的App

```
2
App1 1 09:00 10:00
App2 2 11:00 11:30
09:30
```

数据说明如下：

1.  $N$ 行注册数据以空格分隔，四项数依次表示：App名称、优先级、起始时间、结束时间
2. 优先级1~5，数字越大，优先级越高
3. 时间格式 HH:MM，小时和分钟都是两位，不足两位前面补0
4. 起始时间需小于结束时间，否则注册不上
5. 注册信息中的时间段包含起始时间点，不包含结束时间点

## 输出描述

输出一个字符串，表示App名称，或NA表示空闲时间

## 备注

- 用例保证时间都介于 00:00 - 24:00 之间；
- 用例保证数据格式都是正确的，不用考虑数据输入行数不够、注册信息不完整、字符串非法、优先级超限、时间格式不正确的问题；

## 用例

|    |                                  |
|----|----------------------------------|
| 输入 | 1<br>App1 1 09:00 10:00<br>09:30 |
| 输出 | App1                             |
| 说明 | App1注册在9点到10点间，9点半可用的应用名是App1    |

|    |                                                                           |
|----|---------------------------------------------------------------------------|
| 输入 | 2<br>App1 1 09:00 10:00<br>App2 2 09:10 09:30<br>09:20                    |
| 输出 | App2                                                                      |
| 说明 | App1和App2的时段有冲突，App2优先级比App1高，注册App2后，系统将App1的注册信息自动注销后，09:20时刻可用应用名是App2 |

|    |                                                        |
|----|--------------------------------------------------------|
| 输入 | 2<br>App1 1 09:00 10:00<br>App2 2 09:10 09:30<br>09:50 |
| 输出 | NA                                                     |
| 说明 | App1被注销后，09:50时刻没有应用注册，因此输出NA                          |

## 题目解析

本题数量级较小，可以考虑暴力破解。

本题每注册一个App\_registering前，都要去和已注册的所有App\_registered进行比较：

- 注册时段是否有冲突？

1. 如果没有冲突，则继续和下一个App\_registered比较

2. 如果有冲突，则比较优先级

2.1. App\_registering 的优先级高于 ( $>$ ) App\_registered，则App\_registered需要被注销，此时不能直接进行注销动作，因为我们需要确保 App\_registering 可以注册后，才能进行注销。

2.2. App\_registering 的优先级不高于 ( $\leq$ ) App\_registered，则App\_registering不能注册，即终止后续比较

本题比较两个App注册时段冲突的方式，可以将App的注册时段的时间点信息，转化为分钟数数值，比如：

10:29

可以转化为：

$10 * 60 + 29 = 629$

这样两个App的时段比较，其实判断两个区间是否有交集，假设两个时段分别为

$[s1, e1]$  和  $[s2, e2]$

只要满足

$s1 \geq e2$  或者  $s2 \geq e1$  即可说明两个区间无交集，注意根据题目说明

注册信息中的时间段包含起始时间点，不包含结束时间点

上面两个区间均为左闭右开区间。

且区间保证左边界 $>$ 右边界，因为题目说明

起始时间需小于结束时间，否则注册不上



即，一个App是否可以注册需要先判断其左边界是否大于右边界

2024.03.17

解法一（上面解析的逻辑）中，我们基于输入顺序来注册App，此时“后注册的App”存在一个问题，那就是，如果和前面注册过的某个低优先级的App1存在时间冲突，那么此时不能直接武断地注App1，因为存在下面情况：



App1 1 09:00 10:00  
App2 5 10:10 10:30  
App3 3 09:10 10:30

尝试注册App3时，发现和前面注册的App1冲突，且App1优先级更低，但是我们此时不能注销App1，因为App3还和后面App2冲突，但是App2优先级更高，因此App3注册不了。所以我们也不用注销App1。

这样的话，逻辑就很复杂了。

解法二，优化点在于，提前将输入的App按照优先级降序（对于优先级相同的App，保持输入顺序，即 **稳定排序** ），此时我们会先注册高优先级的App，这样的话，后注册的App只要和前面注册的App发生冲突，则必然无法注册，因为前面已注册App的优先级必然更高。

那么，排序操作，会改变App的注册结果吗？

答案是不会，比如输入的App注册顺序如下



App1 1 09:00 10:00  
App2 5 10:10 10:30  
App3 3 09:10 10:30

按照优先级降序后，变为了



App2 5 10:10 10:30  
App3 3 09:10 10:30

此时注册结果不变，因为我们可以发现：

- 如果不存在App\_low和高优先级App\_high有冲突，则高优先级App\_high提前注册对结果无影响
- 如果存在App\_low和高优先级App\_high有冲突，则无论App\_high在App\_low前面注册，还是在App\_low后面注册，结果都是App\_low无法注册

## JS算法源码

### 解法一

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  class App {
6    constructor(name, priority, start, end) {
7      this.name = name;
8      this.priority = priority;
9      this.start = start;
10     this.end = end;
11   }
12 }
13
14 function convert(time) {
15   // 时间格式 HH:MM，小时和分钟都是两位，不足两位前面补0
16   const [hours, minutes] = time.split(":").map(Number);
17   return hours * 60 + minutes;
18 }
19
20 void (async function () {
21   const n = parseInt(await readline());
22
23   // 需要注册的App
24   const apps = [];
```

[运行](#)

```
25 | for (let i = 0; i < n; i++) {26 |     const [name, priority, startTime, endTime] = (await readline()).split(" ");
27 |     apps.push(
28 |         new App(name, parseInt(priority), convert(startTime), convert(endTime))
29 |     );
30 | }
31 |
32 | // 需要查询的时间点
33 | const queryTime = convert(await readline());
34 |
35 | console.log(getResult(apps, queryTime));
36 | })();
37 |
38 | function getResult(apps, queryTime) {
39 |     // 记录已注册的App
40 |     const registereds = [];
41 |
42 |     outer: for (let app of apps) {
43 |         // 起始时间>=结束时间，则注册不上
44 |         if (app.start >= app.end) continue;
45 |
46 |         // 记录已注册的App中被注销的App的位置
47 |         const idxs = [];
48 |
49 |         // 比较app_registering和它前面完成注册的所有app_registered
50 |         for (let j = 0; j < registereds.length; j++) {
51 |             const registered = registereds[j];
52 |
53 |             // 两个App的注册时间无冲突，则继续和下一个app_registered比较
54 |             if (registered.start >= app.end || app.start >= registered.end) {
55 |                 continue;
56 |             }
57 |
58 |             // 两个App的注册时间有冲突，则比较优先级
59 |             if (app.priority > registered.priority) {
60 |                 // 这里不能直接注销低优先级的app_registered，我们要保证app_registering一定能注册后才能进行此操作，因此先记录要被注销的app_registered的位置
61 |                 idxs.push(j);
62 |             } else {
63 |                 // app_registering的优先级 <= app_registered的优先级时，则app_registering不能注册，终止比较
```



```

64         continue outer;
65     }
66 }
67
68 // idxs中索引是升序的，如果正序删除的话，那么每删除一个元素，都会改变后面元素的索引位置，因此这里采用倒序删除
69 for (let i = idxs.length - 1; i >= 0; i--) {
70     const deleteIdx = idxs[i];
71     registereds.splice(deleteIdx, 1);
72 }
73
74 registereds.push(app);
75 }
76
77 let ans = "NA";
78
79 for (let app of registereds) {
80     if (queryTime >= app.start && queryTime < app.end) {
81         ans = app.name;
82         break;
83     }
84 }
85
86 return ans;
87 }

```

## 解法二

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  class App {
6      constructor(name, priority, start, end) {
7          this.name = name;
8          this.priority = priority;
9          this.start = start;

```

运行

```
10     this.end = end;11 }
12 }
13
14 function convert(time) {
15     // 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
16     const [hours, minutes] = time.split(":").map(Number);
17     return hours * 60 + minutes;
18 }
19
20 void (async function () {
21     const n = parseInt(await readline());
22
23     // 需要注册的App
24     const apps = [];
25     for (let i = 0; i < n; i++) {
26         const [name, priority, startTime, endTime] = (await readline()).split(" ");
27         apps.push(
28             new App(name, parseInt(priority), convert(startTime), convert(endTime))
29         );
30     }
31
32     // 需要查询的时间点
33     const queryTime = convert(await readline());
34
35     console.log(getResult(apps, queryTime));
36 })();
37
38 function getResult(apps, queryTime) {
39     // 记录已注册的App
40     const registereds = [];
41
42     // 按照注册优先级降序
43     apps.sort((a, b) => b.priority - a.priority);
44
45     outer: for (let app of apps) {
46         // 起始时间>=结束时间, 则注册不上
47         if (app.start >= app.end) continue;
48     }
```

```

49   for (let registered of registereds) {
50       // 如果和前面注册的App注册时间有交集（冲突），由于已经按照优先级排序，因此后注册的优先级肯定不比前面高，因此发生冲突时，后注册的无法成功
51       if (hasInterSection(app, registered)) {
52           continue outer;
53       }
54   }
55
56   // 如果和前面高优先级的App无注册时间冲突，则可以注册
57   registereds.push(app);
58 }
59
60 let ans = "NA";
61
62 for (let app of registereds) {
63     if (queryTime >= app.start && queryTime < app.end) {
64         ans = app.name;
65         // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
66         break;
67     }
68 }
69
70 return ans;
71 }
72
73 // 判断两个范围是否有交集
74 function hasInterSection(range1, range2) {
75     const { start: s1, end: e1 } = range1;
76     const { start: s2, end: e2 } = range2;
77
78     if (s1 >= s2 && s1 < e2) {
79         return true;
80     }
81
82     if (s2 >= s1 && s2 < e1) {
83         return true;
84     }
85
86     return false;

```

```
87 | }
```

## Java算法源码

### 解法一

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class Main {
5      static class App {
6          String name;
7          int priority;
8          int startTime;
9          int endTime;
10
11         public App(String name, int priority, int startTime, int endTime) {
12             this.name = name;
13             this.priority = priority;
14             this.startTime = startTime;
15             this.endTime = endTime;
16         }
17     }
18
19     public static void main(String[] args) {
20         Scanner sc = new Scanner(System.in);
21
22         int n = sc.nextInt();
23
24         // 需要注册的App
25         ArrayList<App> apps = new ArrayList<>();
26         for (int i = 0; i < n; i++) {
27             apps.add(new App(sc.next(), sc.nextInt(), convert(sc.next()), convert(sc.next())));
28         }
29     }
```

```

30 // 需要查询的时间点
31         int queryTime = convert(sc.next());
32
33     System.out.println(getResult(apps, queryTime));
34 }
35
36 public static String getResult(ArrayList<App> apps, int queryTime) {
37     // 记录已注册的App
38     ArrayList<App> registereds = new ArrayList<>();
39
40     outer:
41     for (App app : apps) {
42         // 起始时间>=结束时间，则注册不上
43         if (app.startTime >= app.endTime) continue;
44
45         // 记录已注册的App中被注销的App的位置
46         ArrayList<Integer> idxs = new ArrayList<>();
47
48         // 比较app_registering和它前面完成注册的所有app_registered
49         for (int j = 0; j < registereds.size(); j++) {
50             App registered = registereds.get(j);
51
52             // 两个App的注册时间无冲突，则继续和下一个app_registered比较
53             if (registered.startTime >= app.endTime || app.startTime >= registered.endTime) continue;
54
55             // 两个App的注册时间有冲突，则比较优先级
56             if (app.priority > registered.priority) {
57                 // 这里不能直接注销低优先级的app_registered，我们要保证app_registering一定能注册后才能进行此操作，因此先记录要被注销的app_registered的位置
58                 idxs.add(j);
59             } else {
60                 // app_registering的优先级 <= app_registered的优先级时，则app_registering不能注册，终止比较
61                 continue outer;
62             }
63         }
64
65         // idxs中索引是升序的，如果正序删除的话，那么每删除一个元素，都会改变后面元素的索引位置，因此这里采用倒序删除
66         for (int j = idxs.size() - 1; j >= 0; j--) {
67             int deleteIdx = idxs.get(j);

```

```

68         registereds.remove(deleteIdx);69     }
69
70
71     registereds.add(app);
72 }
73
74 String ans = "NA";
75
76 // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
77 for (App app : registereds) {
78     if (queryTime >= app.startTime && queryTime < app.endTime) {
79         ans = app.name;
80         break;
81     }
82 }
83
84 return ans;
85 }
86
87 public static int convert(String time) {
88     // 时间格式 HH:MM，小时和分钟都是两位，不足两位前面补0
89     String[] tmp = time.split(":");
90
91     String hours = tmp[0];
92     String minutes = tmp[1];
93     return Integer.parseInt(hours) * 60 + Integer.parseInt(minutes);
94 }
95 }

```

## 解法二

```

1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 class App {
5     String name;
6     int priority;
7     int startTime;

```

```

8      int endTime;
9
10     public App(String name, int priority, int startTime, int endTime) {
11         this.name = name;
12         this.priority = priority;
13         this.startTime = startTime;
14         this.endTime = endTime;
15     }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         Scanner sc = new Scanner(System.in);
21
22         int n = sc.nextInt();
23
24         // 需要注册的App
25         ArrayList<App> apps = new ArrayList<>();
26         for (int i = 0; i < n; i++) {
27             apps.add(new App(sc.next(), sc.nextInt(), convert(sc.next()), convert(sc.next())));
28         }
29
30         // 需要查询的时间点
31         int queryTime = convert(sc.next());
32
33         System.out.println(getResult(apps, queryTime));
34     }
35
36     public static String getResult(ArrayList<App> apps, int queryTime) {
37         // 记录已注册的App
38         ArrayList<App> registereds = new ArrayList<>();
39
40         // 按照注册优先级降序
41         apps.sort((a, b) -> b.priority - a.priority);
42
43         outer:
44         for (App app : apps) {
45             // 起始时间>=结束时间，则注册不上
46             if (app.startTime >= app.endTime) continue;

```

```
47
48     for (App registered : registereds) {
49         // 如果和前面注册的App注册时间有交集（冲突），由于已经按照优先级排序，因此后注册的优先级肯定不比前面高，因此发生冲突时，后注册的无法成功
50         if (hasInterSection(app, registered)) {
51             continue outer;
52         }
53     }
54
55     // 如果和前面高优先级的App无注册时间冲突，则可以注册
56     registereds.add(app);
57 }
58
59 String ans = "NA";
60
61 for (App app : registereds) {
62     if (queryTime >= app.startTime && queryTime < app.endTime) {
63         ans = app.name;
64         // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
65         break;
66     }
67 }
68
69 return ans;
70 }
71
72 public static int convert(String time) {
73     // 时间格式 HH:MM，小时和分钟都是两位，不足两位前面补0
74     String[] tmp = time.split(":");
75
76     String hours = tmp[0];
77     String minutes = tmp[1];
78     return Integer.parseInt(hours) * 60 + Integer.parseInt(minutes);
79 }
80
81 // 判断两个范围是否有交集
82 private static boolean hasInterSection(App app1, App app2) {
83     int s1 = app1.startTime, e1 = app1.endTime;
84     int s2 = app2.startTime, e2 = app2.endTime;
```



```
85 |      86 |          if (s1 >= s2 && s1 < e2) {
87 |              return true;
88 |          }
89 |
90 |          if (s2 >= s1 && s2 < e1) {
91 |              return true;
92 |          }
93 |
94 |          return false;
95 |      }
96 | }
```

## Python算法源码

### 解法一

```
1 | class App:
2 |     def __init__(self, name, priority, start, end):
3 |         self.name = name
4 |         self.priority = priority
5 |         self.start = start
6 |         self.end = end
7 |
8 |
9 |     def convert(time):
10 |         # 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
11 |         hours, minutes = map(int, time.split(":"))
12 |         return hours * 60 + minutes
13 |
14 |
15 | # 输入获取
16 | n = int(input())
17 |
18 | # 需要注册的App
19 | apps = []
```

```
20 for _ in range(n):21     name, priority, start, end = input().split(" ")
22     apps.append(App(name, int(priority), convert(start), convert(end)))
23
24 # 需要查询的时间点
25 queryTime = convert(input())
26
27
28 # 算法入口
29 def getResult():
30     registereds = []
31
32     for app in apps:
33         # 起始时间>=结束时间，则注册不上
34         if app.start >= app.end:
35             continue
36
37         # 记录已注册的App中被注销的App的位置
38         idxs = []
39
40         flag = False
41
42         # 比较app_registering和它前面完成注册的所有app_registered
43         for j in range(len(registereds)):
44             registered = registereds[j]
45
46             # 两个App的注册时间无冲突，则继续和下一个app_registered比较
47             if registered.start >= app.end or app.start >= registered.end:
48                 continue
49
50             # 两个App的注册时间有冲突，则比较优先级
51             if app.priority > registered.priority:
52                 # 这里不能直接注销低优先级的app_registered，我们要保证app_registering一定能注册后才能进行此操作，因此先记录要被注销的app_registered的位置
53                 idxs.append(j)
54             else:
55                 # app_registering的优先级 <= app_registered的优先级时，则app_registering不能注册，终止比较
56                 flag = True
57                 break
58
```

```

59 | # 如果app_registering不能注册，则终止比较
60 |         if flag:
61 |             continue
62 |
63 | # idxs中索引是升序的，如果正序删除的话，那么每删除一个元素，都会改变后面元素的索引位置，因此这里采用倒序删除
64 | idxs.reverse()
65 | for idx in idxs:
66 |     registereds.pop(idx)
67 |
68 | registereds.append(app)
69 |
70 | ans = "NA"
71 |
72 | # 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
73 | for registered in registereds:
74 |     if registered.start <= queryTime < registered.end:
75 |         ans = registered.name
76 |         break
77 |
78 | return ans
79 |
80 |
81 | # 算法调用
82 | print(getResult())

```

## 解法二

```

1 | class App:
2 |     def __init__(self, name, priority, start, end):
3 |         self.name = name
4 |         self.priority = priority
5 |         self.start = start
6 |         self.end = end
7 |
8 |
9 | def convert(time):
10 |     # 时间格式 HH:MM，小时和分钟都是两位，不足两位前面补0
11 |     hours, minutes = map(int, time.split(":"))

```

```
12     return hours * 60 + minutes
13
14
15 # 判断两个范围是否有交集
16 def hasInterSection(a, b):
17     if b.start <= a.start < b.end:
18         return True
19
20     if a.start <= b.start < a.end:
21         return True
22
23     return False
24
25
26 # 输入获取
27 n = int(input())
28
29 # 需要注册的App
30 apps = []
31 for _ in range(n):
32     name, priority, start, end = input().split(" ")
33     apps.append(App(name, int(priority), convert(start), convert(end)))
34
35 # 需要查询的时间点
36 queryTime = convert(input())
37
38
39 # 算法入口
40 def getResult():
41     # 记录已注册的App
42     registereds = []
43
44     # 按照注册优先级降序
45     apps.sort(key=lambda x: -x.priority)
46
47     for app in apps:
48         # 起始时间>=结束时间，则注册不上
49         if app.start >= app.end:
```

```

50         continue
51     |
52     conflict = False
53
54     for registered in registereds:
55         # 如果和前面注册的App注册时间有交集（冲突），由于已经按照优先级排序，因此后注册的优先级肯定不比前面高，因此发生冲突时，后注册的无法成功
56         if hasInterSection(app, registered):
57             conflict = True
58             break
59
60     if conflict:
61         continue
62
63     # 如果和前面高优先级的App无注册时间冲突，则可以注册
64     registereds.append(app)
65
66     ans = "NA"
67
68     for registered in registereds:
69         if registered.start <= queryTime < registered.end:
70             ans = registered.name
71             # 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
72             break
73
74     return ans
75
76
77 # 算法调用
78 print(getResult())

```

## C算法源码

### 解法一

```

1 #include <stdio.h>
2 #include <stdlib.h>

```

运行

```
3 | #include <string.h> 4 |
5 | #define MAX_SIZE 100
6 |
7 | typedef struct App {
8 |     char name[100];
9 |     int priority;
10 |    int start;
11 |    int end;
12 | } App;
13 |
14 | App *new_App(char *name, int priority, int start, int end) {
15 |     App *app = (App *) malloc(sizeof(App));
16 |
17 |     strcpy(app->name, name);
18 |     app->priority = priority;
19 |     app->start = start;
20 |     app->end = end;
21 |
22 |     return app;
23 | }
24 |
25 | int convert(char *time) {
26 |     int hours;
27 |     int minutes;
28 |     // 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
29 |     sscanf(time, "%d:%d", &hours, &minutes);
30 |     return hours * 60 + minutes;
31 | }
32 |
33 | App *apps[MAX_SIZE];
34 | int apps_size = 0;
35 |
36 | void getResult(int query) {
37 |     // 记录已注册的App
38 |     App *registeds[MAX_SIZE];
39 |     int registeds_size = 0;
40 |
41 |     for (int i = 0; i < apps_size; i++) {
```

```

42 App *app = apps[i];
43
44 // 起始时间>=结束时间，则注册不上
45 if (app->start >= app->end) continue;
46
47 // 记录已注册的App中被注销的App的位置
48 int idxs[MAX_SIZE];
49 int idxs_size = 0;
50
51 int flag = 0;
52
53 // 比较app_registering和它前面完成注册的所有app_registered
54 for (int j = 0; j < registereds_size; j++) {
55     App *registerd = registereds[j];
56
57     // 79~80行会产生NULL的情况
58     if(registerd == NULL) continue;
59
60     // 两个App的注册时间无冲突，则继续和下一个app_registered比较
61     if (registerd->start >= app->end || app->start >= registerd->end) continue;
62
63     // 两个App的注册时间有冲突，则比较优先级
64     if (app->priority > registerd->priority) {
65         // 这里不能直接注销低优先级的app_registered，我们要保证app_registering一定能注册后才能进行此操作，因此先记录要被注销的app_registered的位置
66         idxs[idxs_size++] = j;
67     } else {
68         // app_registering的优先级 <= app_registered的优先级时，则app_registering不能注册，终止比较
69         flag = 1;
70         break;
71     }
72 }
73
74 // app_registering不能注册，终止比较
75 if (flag) continue;
76
77 // 将被注销的App赋值为NULL
78 for (int j = 0; j < idxs_size; j++) {
79     int deleteIdx = idxs[j];
80     registereds[deleteIdx] = NULL;

```

```

81     }
82     |
83     registereds[registered_size++] = app;
84 }
85
86 char *ans = "NA";
87
88 // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
89 for (int i = 0; i < registereds_size; i++) {
90     if (registereds[i] == NULL) continue;
91
92     if(query >= registereds[i]->start && query < registereds[i]->end) {
93         ans = registereds[i]->name;
94         break;
95     }
96 }
97
98 puts(ans);
99 }
100
101 int main() {
102     int n;
103     scanf("%d", &n);
104
105     for (int i = 0; i < n; i++) {
106         // App名称、优先级、起始时间、结束时间
107         char name[100];
108         int priority;
109         char startTime[10];
110         char endTime[10];
111
112         scanf("%s %d %s %s", name, &priority, startTime, endTime);
113
114         int start = convert(startTime);
115         int end = convert(endTime);
116
117         apps[apps_size++] = new_App(name, priority, start, end);
118     }

```



```
119 | 120 | // 需要查询的时间点
121 |     char queryTime[10];
122 |     scanf("%s", queryTime);
123 |
124 |     int query = convert(queryTime);
125 |
126 |     getResult(query);
127 |
128 |     return 0;
129 | }
```

## 解法二

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAX_SIZE 100
6
7 typedef struct App {
8     char name[100];
9     int priority;
10    int start;
11    int end;
12    int id;
13 } App;
14
15 App *new_App(char *name, int priority, int start, int end, int id) {
16     App *app = (App *) malloc(sizeof(App));
17
18     strcpy(app->name, name);
19     app->priority = priority;
20     app->start = start;
21     app->end = end;
22     app->id = id;
23 }
```

```
24     return app;
25 }
26
27 int convert(char *time) {
28     int hours;
29     int minutes;
30     // 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
31     sscanf(time, "%d:%d", &hours, &minutes);
32     return hours * 60 + minutes;
33 }
34
35 int cmp(const void *a, const void *b) {
36     App *A = *(App **) a;
37     App *B = *(App **) b;
38
39     if (A->priority == B->priority) {
40         return A->id - B->id;
41     } else {
42         return B->priority - A->priority;
43     }
44 }
45
46 App *apps[MAX_SIZE];
47 int apps_size = 0;
48
49 void getResult(int query) {
50     // 记录已注册的App
51     App *registered[MAX_SIZE];
52     int registered_size = 0;
53
54     // 按照注册优先级降序
55     qsort(apps, apps_size, sizeof(apps[0]), cmp);
56
57     for (int i = 0; i < apps_size; i++) {
58         App *app = apps[i];
59
60         // 起始时间>=结束时间, 则注册不上
61         if (app->start >= app->end) continue;
```

```

62 |         int conflict = 0;
63 |
64 |
65 |         // 比较app_registering和它前面完成注册的所有app_registered
66 |         for (int j = 0; j < registereds_size; j++) {
67 |             App *registered = registereds[j];
68 |
69 |             // 如果和前面注册的App注册时间有交集（冲突），由于已经按照优先级排序，因此后注册的优先级肯定不比前面高，因此发生冲突时，后注册的无法成功
70 |             if (app->start <= registered->start && registered->start < app->end) {
71 |                 conflict = 1;
72 |                 break;
73 |             }
74 |
75 |             if (registered->start <= app->start && app->start < registered->end) {
76 |                 conflict = 1;
77 |                 break;
78 |             }
79 |         }
80 |
81 |         // 如果和前面高优先级的App无注册时间冲突，则可以注册
82 |         if (!conflict) {
83 |             registereds[registered_size++] = app;
84 |         }
85 |     }
86 |
87 |     char *ans = "NA";
88 |
89 |     for (int i = 0; i < registereds_size; i++) {
90 |         if (registereds[i] == NULL) continue;
91 |
92 |         if (query >= registereds[i]->start && query < registereds[i]->end) {
93 |             ans = registereds[i]->name;
94 |             // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
95 |             break;
96 |         }
97 |     }
98 |
99 |     puts(ans);
100 | }

```

```

101 |
102 | int main() {
103 |     int n;
104 |     scanf("%d", &n);
105 |
106 |     for (int i = 0; i < n; i++) {
107 |         // App名称、优先级、起始时间、结束时间
108 |         char name[100];
109 |         int priority;
110 |         char startTime[10];
111 |         char endTime[10];
112 |
113 |         scanf("%s %d %s %s", name, &priority, startTime, endTime);
114 |
115 |         int start = convert(startTime);
116 |         int end = convert(endTime);
117 |
118 |         apps[apps_size++] = new_App(name, priority, start, end, i);
119 |     }
120 |
121 |     // 需要查询的时间点
122 |     char queryTime[10];
123 |     scanf("%s", queryTime);
124 |
125 |     int query = convert(queryTime);
126 |
127 |     getResult(query);
128 |
129 |     return 0;
130 | }

```

## C++算法源码

### 解法一

```

1 | #include <bits/stdc++.h>

```

```
2 using namespace std; 3 |
4 class App {
5 public:
6     string name;
7     int priority;
8     int startTime;
9     int endTime;
10
11     App(string name, int priority, int startTime, int endTime) : name(std::move(name)), priority(priority),
12                                                                    startTime(startTime), endTime(endTime) {};
13 };
14
15 int convert(string &s) {
16     // 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
17     stringstream ss(s);
18
19     string hours;
20     getline(ss, hours, ':');
21
22     string minutes;
23     getline(ss, minutes, ':');
24
25     return stoi(hours) * 60 + stoi(minutes);
26 }
27
28 string solution(vector<App> &apps, int queryTime) {
29     // 记录已注册的App
30     vector<App> registereds;
31
32     for (const auto &app: apps) {
33         // 起始时间>=结束时间, 则注册不上
34         if (app.startTime >= app.endTime) continue;
35
36         // 记录已注册的App中被注销的App的位置
37         vector<int> idxs;
38
39         bool flag = false;
40
```

```

41 // 比较app_registering和它前面完成注册的所有app_registered
42 |
43 App registered = registereds[j];
44
45 // 两个App的注册时间无冲突，则继续和下一个app_registered比较
46 if (registered.startTime >= app.endTime || app.startTime >= registered.endTime) {
47     continue;
48 }
49
50 // 两个App的注册时间有冲突，则比较优先级
51 if (app.priority > registered.priority) {
52     // 这里不能直接注销低优先级的app_registered，我们要保证app_registering一定能注册后才能进行此操作，因此先记录要被注销的app_registered的位置
53     idxs.emplace_back(j);
54 } else {
55     // app_registering的优先级 <= app_registered的优先级时，则app_registering不能注册，终止比较
56     flag = true;
57 }
58 }
59
60 if (flag) {
61     continue;
62 }
63
64 // idxs中索引是升序的，如果正序删除的话，那么每删除一个元素，都会改变后面元素的索引位置，因此这里采用倒序删除
65 for (int j = idxs.size() - 1; j >= 0; j--) {
66     int deleteIdx = idxs[j];
67     registereds.erase(registereds.begin() + deleteIdx);
68 }
69
70 registereds.emplace_back(app);
71 }
72
73 string ans = "NA";
74
75 // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
76 for (const auto &app: registereds) {
77     if (queryTime >= app.startTime && queryTime < app.endTime) {
78         ans = app.name;
79         break;

```

```

80     }
81 }
82
83     return ans;
84 }
85
86 int main() {
87     int n;
88     cin >> n;
89
90     // 需要注册的App
91     vector<App> apps;
92     for (int i = 0; i < n; i++) {
93         string name;
94         int priority;
95         string startTime;
96         string endTime;
97
98         cin >> name >> priority >> startTime >> endTime;
99
100        apps.emplace_back(name, priority, convert(startTime), convert(endTime));
101    }
102
103    // 需要查询的时间点
104    string queryTime;
105    cin >> queryTime;
106
107    cout << solution(apps, convert(queryTime)) << endl;
108
109    return 0;
110 }

```

## 解法二

```

1  #include <bits/stdc++.h>
2
3  using namespace std;

```

```
4 | 5 | class App {
6 | public:
7 |     string name;
8 |     int priority;
9 |     int startTime;
10 |    int endTime;
11 |
12 |    App(string name, int priority, int startTime, int endTime) : name(std::move(name)), priority(priority),
13 |                                                                startTime(startTime), endTime(endTime) {};
14 | };
15 |
16 | int convert(string &s) {
17 |     // 时间格式 HH:MM, 小时和分钟都是两位, 不足两位前面补0
18 |     stringstream ss(s);
19 |
20 |     string hours;
21 |     getline(ss, hours, ':');
22 |
23 |     string minutes;
24 |     getline(ss, minutes, ':');
25 |
26 |     return stoi(hours) * 60 + stoi(minutes);
27 | }
28 |
29 | string solution(vector<App> &apps, int queryTime) {
30 |     // 记录已注册的App
31 |     vector<App> registereds;
32 |
33 |     // 按照注册优先级降序(注意对于相同优先级的App, 需要维持原有顺序, 因此需要稳定排序)
34 |     stable_sort(apps.begin(), apps.end(), [](const App& app1, const App& app2) {
35 |         return app1.priority > app2.priority;
36 |     });
37 |
38 |     for (const auto &app: apps) {
39 |         // 起始时间>=结束时间, 则注册不上
40 |         if (app.startTime >= app.endTime) continue;
41 |
42 |         bool conflict = false;
```



```

43 |
44 |         for (const auto &registered: registereds) {
45 |             // 如果和前面注册的App注册时间有交集（冲突），由于已经按照优先级排序，因此后注册的优先级肯定不比前面高，因此发生冲突时，后注册的无法成功
46 |             if ((app.startTime <= registered.startTime && registered.startTime < app.endTime) ||
47 |                 (registered.startTime <= app.startTime && app.startTime < registered.endTime)) {
48 |                 conflict = true;
49 |                 break;
50 |             }
51 |         }
52 |
53 |         // 如果和前面高优先级的App无注册时间冲突，则可以注册
54 |         if (!conflict) {
55 |             registereds.emplace_back(app);
56 |         }
57 |     }
58 |
59 |     string ans = "NA";
60 |
61 |     for (const auto &app: registereds) {
62 |         if (queryTime >= app.startTime && queryTime < app.endTime) {
63 |             ans = app.name;
64 |             // 注册成功的App时段之间互不冲突，因此queryTime只会对应一个App
65 |             break;
66 |         }
67 |     }
68 |
69 |     return ans;
70 | }
71 |
72 | int main() {
73 |     int n;
74 |     cin >> n;
75 |
76 |     // 需要注册的App
77 |     vector<App> apps;
78 |     for (int i = 0; i < n; i++) {
79 |         string name;
80 |         int priority;

```

```
81 |         string startTime;82 |         string endTime;
83 |
84 |         cin >> name >> priority >> startTime >> endTime;
85 |
86 |         apps.emplace_back(name, priority, convert(startTime), convert(endTime));
87 |     }
88 |
89 |     // 需要查询的时间点
90 |     string queryTime;
91 |     cin >> queryTime;
92 |
93 |     cout << solution(apps, convert(queryTime)) << endl;
94 |
95 |     return 0;
96 | }
```